Reinforcement Learning
Pinball PPO Agent

1 - Objectifs :

L’objectif est d’entraîner un agent PPO a jouer au pinball dans une simulation simple. L’agent sera en charge de contrôler un des deux flippers du pinball.

Entraînement par curriculum :

- Viser une cible aléatoire avec une configuration de balle (position /vélocité) initiale régulière sur sessions courtes (une balle = un tir)
- Viser une cible aléatoire avec une configuration de balle (position / vélocité) aléatoire sur sessions courtes (une balle = un tir)
- Jouer une partie longue avec deux flippers, un agents par flipper.

Pour chaque étape du curriculum :

- Déterminer les observations pertinentes pouvant améliorer la qualité des décisions.
- Déterminer les récompenses adaptées à chaque phase

Diagnostique :

- Développer un système de récompenses adapté au diagnostique de l’apprentissage. Permettre de sortir ce diagnostique sous format textuel (ou CSV).
- Rendre paramétrable le système de récompense par fichier de configuration.
- Tout au long du TP, conserver les versions des politiques qui semblent faire avancer l’apprentissage dans un sens voulu. Conserver également le diagnostique et le fichier de paramétrage des récompenses associé.

Correction des comportements :

- Vérifier que le comportement de la politique est visuellement celui attendu.
- Dans le cas de comportements non-souhaités, adapter le système d’observations et de récompenses pour corriger ces comportements.

2 – Premier apprentissage

2.1 – Attribuer une récompense.

Sous Unity :

- Ouvrir la scène « EasySingleFlipperAgentTraining ».
- Sélectionner le game-object « FlipperAgent ».
- Passer son paramètre « Behaviour Type » à la valeur « Heuristique Only » (jouable par un humain). En PlayMode, « Ctrl gauche » contrôlera le flipper de gauche.
- Dans la classe FlipperAgent, émettre un signal de récompense de -1 si la balle touche le drain.
- Afficher en console l'attribution de chaque récompense.
- A la fin d'un épisode, afficher en console le cumul des récompenses (CumulatedRewards).

Lancer le PlayMode et vérifier que les récompenses sont bien attribuées.

2.2 – Ajouter une valeur observée

Dans la classe « FlipperAgent.cs », compléter la fonction d'observation de manière à observer la position de la balle relativement au flipper :

- Calculer le vecteur Flipper→Ball
- Normaliser ce veteur en fonction des dimensions standard d'un pinball. Utiliser : PinballConfiguration.AveragePinballDimension
- Pivoter ce vecteur dans le repère de la raquette à l'état neutre (utiliser : Flipper.WorldToLocalVector)
- Appeler la fonction d'observation de l'agent avec ce vecteur position en paramètres

Sous Unity :

- Ouvrir le prefab « FlipperAgent.prefab ».
- Passer Vector Observation → Space Size de 0 à 2

On observe à présent deux valeurs : B(x,y), la position de la balle dans un repère centré et orienté sur la flipper.

2.3 – Lancer un entraînement

Dans la console Anaconda :

- Activer l'environnement « mlagents » : conda activate mlagents
- Se positionner sur le dossier d'entrainement : cd "...\MachineLearningTP\Training"
- Lancer l'entraînement avec la commande :

```
.\mlagents-learn-copyto.ps1 -config "FirstFlipperPPO.yaml" -initializefrom ""
-runid "FirstFlipper_0_0_0" -copyto "..\Assets\Models\" -resume
```

Sous Unity :

- Ouvrir la scène « EasySingleFlipperMultiAgentTraining »
- Appuyer sur play lorsque la console Anaconda est en attente de connexion.

L'entraînement doit se terminer par une génération de fichier « .onnx »

2.4 – Effectuer une analyser

Sous Unity :

- Ouvrir la scène « EasySingleFlipperAgentTraining »
- Sélectionner le game-object « FlipperAgent ».
- Passer son paramètre « Behaviour Type » à la valeur « Inference Only » (jouable par une policy).
- Sélectionner dans « Model » la référence au modèle entraîné dans « Assets / Models »
- Appuyer sur Play

Vérifier que l'agent a bien appris à éviter la pénalité liée à la perte de la balle dans le drain.

Dans la console Anaconda :

- Lancer TensorBoard : tensorboard --logdir results --port 6006
- Ouvrir l'interface de TensorBoard : « Ctrl + Clic gauche » sur le lien en résultat de console, ou ouvrir http://localhost:6006/ dans un navigateur web

Dans la fenêtre TensorBoard :

- Dans l'onglet Time Series, observer les courbes « Cumulative Reward » et « Losses »
- Déplier tout en bas, Policies et observer la courbe « Entropy »

Les courbes doivent toutes converger vers une sorte de plateau.

- Cumulative reward doit se rapprocher de l'espérence de gain par rapport aux récompenses. Dans notre cas, la courbe doit converger vers 0.
- Losses doit tendre vers 0 sans jamais l'atteindre. Environ 0,05 pour notre cas.
- Entropy doit tendre vers 0, en limitant le bruit. Avec peu d'observation, on pourrait s'attendre à un apprentissage sous-optimal et bruité.

Bravo ! Vous avez entraîné votre premier modèle d'IA !

3 – Système de diagnostique

Construire un système d'attribution des récompenses et de diagnostique de sorte que :

- Chaque type de récompense est associé à un enum ou à une string
- Un configuration associe chaque type de récompense à sa valeur.
- Chaque récompense donnée à l'agent est également comptabilisé dans une structure de décompte des récompenses.
- Le système doit avoir les notions suivantes :
  - Moyenne des récompenses passée (fréquence régulière et en fin d'épisode)
  - Cumul des récompenses de l'agent
  - Cumul des récompenses de l'ensemble des agents dans l'environnement0
- Le système doit permettre d'afficher ces données en console ou de les convertir en fichiers text ou csv.

Le système de diagnostique est essentiel pour vérifier que les récompenses données ont eu l'impact souhaité lors de l'entraînement.

C'est le meilleur moyen de savoir si :

- **Une version de la politique est meilleur qu'une autre.**
- **Un comportement en particulier a été appris ou non.**

4 – A vous de jouer !

A vous de trouver les observation et les récompenses adaptées pour passer les étapes du curriculum.

Pour les entraînement suivants, utiliser les configurations PyTorch :

- SingleFlipperPPO.yaml : Entraînement bien plus long que « FirstFlipperPPO.yaml »
- DualFlipperPPO.yaml : Entraînement encore plus long, taille des trajectoire adaptée à des impacts plus éloignés, plus d'entropie pour favoriser la découverte sur un plateau très différent.

Pour repartir d'un modèle pré-entrainé, utiliser « -initializefrom » :

> .\mlagents-learn-copyto.ps1 -config "FirstFlipperPPO.yaml" -initializefrom **"FirstFlipper_0_0_0" -runid "FirstFlipper_0_0_1"** -copyto "..\Assets\Models\" -resume

Il est conseillé de garder des numéros de version des IA intéressantes.

Si une version ne vous plaît pas, il est possible de l'écraser en réutilisant son identifiant et : -force

Que le meilleur touche le plus de cibles !

Bon TP à toutes et tous !